Relazione Tecnica

Progetto Energia Solare API

Ingegneria dei Sistemi Distribuiti

Link GitHub: https://github.com/CiccioStissi/Energia_solare.git

1. Introduzione e Obiettivi

Il progetto Energia Solare API e' un servizio REST sviluppato in Python per l'interrogazione e la gestione di un dataset di produzione di energia fotovoltaica.

Obiettivi principali

- Esporre un'API REST per l'analisi dei dati di produzione solare (energia kWh, irradiazione W/m2, temperatura).
- Implementare un sistema di autenticazione e autorizzazione basato su ruoli tramite token JWT.
- Gestire l'importazione di file CSV in modo asincrono e disaccoppiato tramite RabbitMQ.
- Applicare i pattern → Remote Facade, DTO, Request Batch, Client Session State, Token JWT, RabbitMQ.

2. Architettura Generale

Il sistema e' composto da tre processi indipendenti che collaborano attraverso protocolli standard con un database PostgreSQL condiviso.

Componenti

Componente	Tecnologia	Ruolo
Client	test_all.py	Menu CLI interattivo. Invia richieste HTTP, salva JWT in. energia_token tra le sessioni.
Server	FastAPI + uvicorn	API REST asincrona. Autenticazione, query analitiche, ricezione CSV.
Message Broker	RabbitMQ	Riceve job CSV dal server e li consegna al worker.
Worker	csv_worker.py	Consuma la coda, elabora CSV con pandas, aggiorna il DB.
Database	PostgreSQL	Tabelle: users, solar_production, import_jobs.

Schema di comunicazione

```
+-------------------+     HTTP      +-------------------------------+
|  client/          | -----------> |  FastAPI Server (main.py)     |
|  test_all.py      | <----------- |  porta 8000                   |
+-------------------+               +----------------+-------------+
                                                     |
                                        pubblica messaggio JSON
                                                     |
                                                     v
                                    +--------------------------------+
                                    |  RabbitMQ (broker)             |
                                    |  coda: csv_import  porta 5672  |
                                    +----------------+---------------+
                                                     |
                                          consuma messaggio
                                                     |
                                                     v
                                    +--------------------------------+
                                    |  Worker (csv_worker.py)        |
                                    |  processo separato             |
                                    +----------------+---------------+
                                                     |
                                    +----------------v---------------+
                                    |  PostgreSQL (DB condiviso)     |
                                    +--------------------------------+
```

3. Stack Tecnologico

Componente	Tecnologia	Note
Web framework	FastAPI	Framework ASGI asincrono, auto-documentazione Swagger
ORM	SQLAlchemy (async)	Modalita' asincrona con asyncpg
Database	PostgreSQL	Engine relazionale
Driver DB	asyncpg	Driver asincrono nativo Python
Message Broker	RabbitMQ	Protocollo AMQP 0-9-1
Client AMQP	aio-pika	Client asincrono per RabbitMQ
Autenticazione	JWT	Algoritmo HS256
Hashing password	bcrypt	One-way hashing
Validazione config	pydantic-settings	Lettura .env con validazione tipi
Elaborazione dati	pandas	Parsing CSV, normalizzazione colonne
Server ASGI	uvicorn	Server di produzione
Client HTTP	requests	Usato dal client CLI

4. Struttura del Progetto

```
energia_solare/
├── client/
│   ├── test_all.py                    ← client interattivo da terminale
│   └── .energia_token                 ← token JWT salvato dopo il login
├── data/
│   ├── normalize_csv.py
│   ├── solar_production.csv
│   └── solar_production_normalized.csv
├── server/
│   ├── main.py                        ← [Template Method] lifespan:
startup → run → shutdown
│   ├── config.py                      ← [Singleton] settings = Settings()
│   ├── database.py                    ← [Singleton] engine · [Factory]
async_sessionmaker
│   ├── rabbitmq.py                    ← [Message Queue] connect / publish
/ close
│   ├── .env                           ← variabili d'ambiente (SECRET_KEY,
DB_URL, RABBITMQ_URL, ...)
│   ├── requirements.txt
│   ├── auth/
│   │   ├── __init__.py
│   │   ├── jwt.py                     ← [Token JWT] create_access_token /
decode_token
│   │   └── dependencies.py            ← [Token JWT] guard · [DI] · [Chain
of Responsibility]
│   |                                   bearer → decode → DB lookup →
role check
│   ├── models/
│   │   ├── __init__.py
│   │   ├── user.py
│   │   ├── production.py
│   │   └── job.py                     ← ImportJob: traccia stato dei job
di importazione CSV
│   ├── schemas/
│   │   ├── __init__.py
│   │   ├── auth.py                    ← [DTO] LoginRequest · TokenResponse
│   │   ├── user.py                    ← [DTO] UserInfo (nasconde
hashed_password)
│   │   ├── production.py              ← [DTO] ProductionRecord ·
SuggestionsResponse · ...
│   │   └── job.py                     ← [DTO] JobStatusResponse
│   ├── routers/
│   │   ├── __init__.py
│   │   ├── auth.py                    ← [Remote Facade] POST /auth/login ·
/register
│   │   ├── production.py              ← [Remote Facade] GET /production/*
│   │   └── admin.py                   ← [Remote Facade] POST
/admin/upload-csv (202) · GET /admin/job-status/{id}
│   ├── services/
│   │   ├── __init__.py
│   │   ├── production.py              ← [Service Layer] · [Request Batch]
suggestions()
│   │   └── csv_importer.py            ← [Service Layer]
│   ├── workers/
```

```
|    |    ├──── __init__.py
|    |    └──── csv_worker.py              ← [Message Queue] consumer RabbitMQ
- elabora i CSV in modo asincrono
|    └──── uploads/                       ← file CSV temporanei in attesa di
elaborazione
└──── moment.txt+-- README.md
```

5. Componenti Principali

5.1 Configurazione — config.py

La configurazione e' centralizzata in una classe Settings (pydantic-settings). Legge le variabili dal file server/.env e le valida con i tipi dichiarati al momento dell'importazione. Se una variabile obbligatoria manca, il server crasha immediatamente. L'istanza settings e' un Singleton globale condiviso da tutti i moduli.

```
class Settings(BaseSettings):
    DATABASE_URL: str
    SECRET_KEY: str
    ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 60
    FIRST_ADMIN_PASSWORD: str
    RABBITMQ_URL: str = "amqp://guest:guest@localhost/"
    model_config = SettingsConfigDict(env_file=".env")

settings = Settings()   # Singleton globale
```

5.2 Modelli ORM

Il database contiene tre tabelle:

- **User (tabella: users):** id PK, username UNIQUE indexed, hashed_password, role (user|admin)
- **SolarProduction (tabella: solar_production):** id PK, timestamp UNIQUE indexed, energy_kwh, radiation_wm2 (nullable), temperature_c (nullable).
- **ImportJob (tabella: import_jobs):** id UUID PK, filename, status (queued->processing->done|failed), rows_imported, error, created_at, updated_at. [Traccia il ciclo di vita di ogni job CSV]

5.3 Schemi DTO

I modelli Pydantic in schemas/ fungono da Data Transfer Object ovvero separano la rappresentazione interna ORM dalle richieste HTTP. La validazione automatica garantisce conformita' allo schema per input e output.

Schema	Scopo
LoginRequest / RegisterRequest	Riceve username e password nelle richieste auth
TokenResponse	Restituisce access_token e token_type
ProductionRecord	Singola lettura del sensore fotovoltaico
TopEntry	Entry per le classifiche (top-hours, top-radiation)
HourlyStats / MonthlyAggregate	Statistiche aggregate orarie e mensili
AveragesResponse / SuggestionsResponse	Risposta aggregata del Request Batch
JobStatusResponse	Stato corrente di un job CSV (senza path interni)

5.5 Autenticazione JWT

Il modulo auth/jwt.py fornisce create_access_token() e decode_token().

Il Payload JWT → sub (username), role, exp (scadenza) e firma HMAC-SHA256 con SECRET_KEY.

```
def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) +
timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode["exp"] = expire
    return jwt.encode(to_encode, settings.SECRET_KEY,
algorithm=settings.ALGORITHM)
```

Le dependency in auth/dependencies.py implementano la catena di autorizzazione (Chain of Responsibility):

HTTPBearer -> decode_token -> DB lookup -> role check. get_current_user: verifica token + utente nel DB (usata da /production/*). require_admin: aggiunge verifica role=="admin" (usata da /admin/*).

5.6 Router (Remote Facade)

Router	Endpoint	Accesso
routers/auth.py	POST /auth/login	Pubblico
	POST /auth/register	Pubblico
routers/production.py	GET /production/top-hours?limit=N	user / admin
	GET /production/top-radiation?limit=N	user / admin
	GET /production/monthly-aggregate	user / admin
	GET /production/averages	user / admin
	GET /production/suggestions	user / admin
routers/admin.py	POST /admin/upload-csv	admin
	GET /admin/job-status/{id}	admin

Ogni router delega interamente la logica al Service Layer.

5.7 Service Layer

Metodo	Aggregazione SQL	Output
top_hours(db, limit)	GROUP BY ora, ORDER BY avg(energy_kwh) DESC	Ore con piu' produzione media
top_radiation(db, limit)	GROUP BY ora, ORDER BY avg(radiation_wm2) DESC	Ore con piu' irradiazione media
monthly_aggregate(db)	GROUP BY anno+mese, ORDER BY anno, mese	Produzione totale per mese
averages(db)	top_hours + monthly_aggregate	Statistiche orarie e mensili combinate
suggestions(db)	Chiama tutti e 4 i metodi	Report completo - Request Batch

CsvImporter (services/csv_importer.py) → normalizza colonne, converte tipi e rimuove righe invalide.

5.8 Message Queue — RabbitMQ

Proprieta' chiave dell'implementazione (rabbitmq.py):

- connect_robust: riconnessione automatica in caso di interruzione temporanea del broker.
- durable=True sulla coda: i messaggi rimangono al riavvio di RabbitMQ.
- DeliveryMode.PERSISTENT: i messaggi sono scritti su disco dal broker
- prefetch_count=10: flow control, max 10 messaggi non confermati contemporaneamente.
- Default exchange con routing_key = nome coda, routing diretto avendo solo un server.

5.9 Worker asincrono — csv_worker.py

Processo standalone indipendente dal server.

Flusso per ogni messaggio:

1. Decodifica messaggio JSON (job_id, file_path).
2. Aggiorna job -> status="processing" nel DB.
3. Legge il CSV da uploads/ e chiama CsvImporter.import_csv().
4. Aggiorna job -> status="done" (rows_imported) oppure "failed" (error).
5. Invia ACK a RabbitMQ.
6. Elimina il file CSV temporaneo da uploads/.

5.10 Client interattivo — test_all.py

Menu CLI a numeri (0-10) che copre tutte le funzionalita' dell'API. Il token JWT e' salvato in .energia_token e riletto automaticamente ad ogni operazione. Timeout 10s su ogni chiamata HTTP con gestione esplicita di requests.exceptions.Timeout. Dopo upload CSV, polling automatico ogni 2s fino a completamento del job.

6. Pattern Implementati

Remote Facade

Definizione: Interfaccia coarse-grained che raggruppa operazioni correlate nascondendo la complessita' interna.

File: `routers/auth.py, routers/production.py, routers/admin.py`

Client Session State

Definizione: Lo stato della sessione e' mantenuto lato client e inviato con ogni richiesta, mantenendo il server stateless.

File: `auth/jwt.py, auth/dependencies.py, client/test_all.py`

DTO (Data Transfer Object)

Definizione: Oggetti serializzabili che disaccoppiano la rappresentazione interna ORM da quella esposta via HTTP.

File: `schemas/auth.py, schemas/production.py, schemas/job.py`

Request Batch

Definizione: Raggruppa piu' richieste logiche in un'unica chiamata di rete per ridurre la latenza da round-trip.

File: `services/production.py -> suggestions(), routers/production.py`

Token JWT

Definizione: Token auto-contenuto e firmato crittograficamente per autenticare e autorizzare senza stato server.

File: `auth/jwt.py, auth/dependencies.py`

Message Queue (RabbitMQ)

Definizione: Comunicazione asincrona e disaccoppiata tra produttori e consumatori tramite broker persistente.

File: `rabbitmq.py, routers/admin.py, workers/csv_worker.py`

8. Endpoint Disponibili

Metodo	Endpoint	Ruolo	Descrizione
POST	/auth/register	pubblico	Registra nuovo utente
POST	/auth/login	pubblico	Login → JWT
GET	/production/top-hours	user/admin	Ore del giorno con più produzione media
GET	/production/top-radiation	user/admin	Ore con più irradiazione media
GET	/production/monthly-aggregate	user/admin	Produzione totale per mese
GET	/production/averages	user/admin	Medie orarie e mensili
GET	/production/suggestions	user/admin	Tutte le analisi in un'unica risposta (Request Batch)

Metodo	Endpoint	Ruolo	Descrizione
POST	/admin/upload-csv	admin	Upload CSV → job asincrono via RabbitMQ
GET	/admin/job-status/{id}	admin	Stato elaborazione job CSV

9. Flussi Operativi

9.1 Avvio del sistema

```
# Installa dipendenze
pip install -r server/requirements.txt

# Avvia tutto (RabbitMQ attivo su porta 5672)
./start.sh
# -> uvicorn (FastAPI porta 8000) + csv_worker in background

# Client
python client/test_all.py

# Swagger UI
http://localhost:8000/docs
```

9.2 Autenticazione e autorizzazione

```
Client                              Server
  |                                    |
  | POST /auth/login                   |
  | { username, password } -------> |
  |                                    | SELECT user WHERE username=...
  |                                    | bcrypt.verify(password, hashed)
  |                                    | create_access_token({sub, role, exp})
  | { "access_token": "eyJ..." } <- |
  | [salva in .energia_token]          |
  |                                    |
  | GET /production/top-hours          |
  | Authorization: Bearer eyJ... -->|
  |                                    | HTTPBearer -> estrae token
  |                                    | decode_token -> verifica firma+exp
  |                                    | SELECT user WHERE username=sub
  |                                    | ProductionService.top_hours(db)
  | [{ "ora": "12:00-13:00", ... }] |
  | <--------------------------------|
```

9.3 Upload CSV asincrono

```
Client         Server (FastAPI)    RabbitMQ      Worker
  |                |                  |             |
  | POST /upload-csv|                 |             |
  | (multipart) -->|                 |             |
  |                | salva CSV disco  |             |
  |                | crea ImportJob   |             |
  |                | publish(job_id) -----------> |
  | 202 {job_id} <-|                 |             |
  |                |                  |  consuma msg|
  | GET /job-status |                 |  job->proc. |
  | -------------->|                 |  CsvImporter|
  | {status:proc.} <|                |             |
  |                |                  |  job->done  |
  | GET /job-status |                 |  ACK ------>|
  | -------------->|                 |             |
  | {status:done,   |                 |             |
  |  rows:8760} <--|                 |             |
```

9.4 Query analitiche

Esempio SQL generato da ProductionService.top_hours():

```
SELECT EXTRACT(hour FROM timestamp)    AS ora,
       AVG(energy_kwh)                 AS avg_energy_kwh,
       SUM(energy_kwh)                 AS total_energy_kwh
FROM   solar_production
GROUP  BY EXTRACT(hour FROM timestamp)
ORDER  BY AVG(energy_kwh) DESC
LIMIT  10;
```

10. Sicurezza

Aspetto	Implementazione
Password storage	bcrypt one-way hashing, mai in chiaro nel DB
Autenticazione	JWT firmato HMAC-SHA256, scadenza configurabile in .env
Autorizzazione	Role-based → user (sola lettura) vs admin (upload CSV)
Username riservato	POST /auth/register blocca username "admin" (HTTP 403)
Enumerazione utenti	Login risponde sempre "Username o password non corretti" (401) indipendentemente dal motivo del fallimento
Credenziali	SECRET_KEY, FIRST_ADMIN_PASSWORD, DATABASE_URL in .env (escluso da git)
Validazione input	Pydantic valida tutti i body in ingresso con schema dichiarato
CSV	ON CONFLICT DO NOTHING (nessun duplicato)
File temporanei	I CSV sono eliminati dal worker dopo elaborazione

11. Conclusioni

Il progetto implementa un servizio REST completo per l'analisi di dati fotovoltaici, applicando concretamente i pattern:

- Remote Façade→ interfaccia HTTP semplice sopra una logica interna strutturata su layer.
- Client Session State→ server stateless, identita' e ruolo portati dal client nel JWT.
- DTO → disaccoppiamento netto tra layer ORM e superficie API tramite Pydantic.
- Request Batch→ riduzione round-trip con GET /production/suggestions.
- Token JWT→ autenticazione e autorizzazione senza stato server, con firma crittografica.
- Message Queue→ disaccoppiamento temporale tra upload CSV ed elaborazione via RabbitMQ.